

SQL: Mini Project

SQL Project – Retail Store

Level 1: Basics

1. Retrieve customer names and emails:

SQL: SELECT name, email FROM customers;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the query: `SELECT name, email FROM customers;`. The Results grid displays the following data:

name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Katlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com

The Output pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
2141	19:25:28	INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (41, 8, 6, 5, 'Great product, love it!')	1 row(s) affected / 0.016 sec	
2142	19:25:28	INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (42, 49, 1, 2, 'Would definitely recommend')	1 row(s) affected / 0.000 sec	
2143	19:25:28	INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (43, 13, 1, 3, 'Would definitely recommend')	1 row(s) affected / 0.000 sec	
2144	19:29:58	SELECT name, email FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec

2. View complete product catalog:

SQL: SELECT * FROM products;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the query: `SELECT * FROM products;`. The Results grid displays the following data:

product_id	name	category	price	stock_quantity	added_on
1	Pile of No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-20 02:40:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42

The Output pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
2142	19:25:28	INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (44, 12, 6, 5, 'Great product, love it!')	1 row(s) affected / 0.000 sec	
2143	19:25:28	INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (45, 26, 17, 2, 'Didn't meet expectations')	1 row(s) affected / 0.000 sec	
2144	19:29:58	SELECT name, email FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2145	19:37:47	SELECT * FROM products LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

3. List all unique product categories:

SQL:

SELECT DISTINCT category FROM products;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
2187 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (45, 26, 17, 2, 'Didn''t meet expect
2188 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (46, 18, 29, 4, 'Good value for mo
2189 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (47, 32, 22, 5, 'Product arrived d
2190 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (48, 43, 28, 1, 'Excellent build q
2191 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (49, 18, 13, 5, 'Good value for mo
2192 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (50, 38, 27, 2, 'Excellent build q
2193 • SELECT name, email FROM customers;
2194 • #2 View complete product catalog:
2195 • SELECT * FROM products;
2196 • #3 List all unique product categories:
2197 • SELECT DISTINCT category FROM products;
```

The Result Grid shows the output of the last query:

category
Home
Clothing
Electronics
Toys
Books

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2143	19:25:28	INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUE...	1 row(s) affected	0.000 sec / 0.000 sec
2144	19:29:58	SELECT name, email FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2145	19:37:47	SELECT * FROM products LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
2146	19:39:50	SELECT DISTINCT category FROM products LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec

4. Show products priced above ₹1,000:

SQL:

SELECT * FROM products WHERE price > 1000;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
2189 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (47, 32, 22, 5, 'Product arrived d
2190 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (48, 43, 28, 1, 'Excellent build d
2191 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (49, 18, 13, 5, 'Good value for mo
2192 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (50, 38, 27, 2, 'Excellent build q
2193 • SELECT name, email FROM customers;
2194 • #2 View complete product catalog:
2195 • SELECT * FROM products;
2196 • #3 List all unique product categories:
2197 • SELECT DISTINCT category FROM products;
2198 • #4 Show products priced above ₹1,000:
2199 • SELECT * FROM products WHERE price > 1000;
```

The Result Grid shows the output of the last query:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indexd	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2144	19:29:58	SELECT name, email FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2145	19:37:47	SELECT * FROM products LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
2146	19:39:58	SELECT DISTINCT category FROM products LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec
2147	19:41:37	SELECT * FROM products WHERE price > 1000 LIMIT 0, 1000	40 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

5. Products within ₹2,000 to ₹5,000:

SQL:

SELECT * FROM products WHERE price BETWEEN 2000 AND 5000;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a series of queries, with the last one being: `SELECT * FROM products WHERE price BETWEEN 2000 AND 5000;`. The Results tab displays the output of this query as a table with 13 rows. The table has columns: product_id, name, category, price, stock_quantity, and added_on.

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.86	36	2025-03-20 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38

6. Fetch data for specific customer IDs (Example 1, 5, 10):

SQL:

SELECT * FROM customers WHERE customer_id IN (1, 5, 10);

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a series of queries, with the last one being: `SELECT * FROM customers WHERE customer_id IN (1, 5, 10);`. The Results tab displays the output of this query as a table with 6 rows. The table has columns: customer_id, name, email, phone, and created_at.

customer_id	name	email	phone	created_at
1	Thomas Owers	user1@example.com	142-479-1945	2024-10-14 16:01:12
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43

SQL: Mini Project

7. Identify customers whose names start with 'A':

SQL:

SELECT * FROM customers WHERE name LIKE 'A%';

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a list of queries, with the last one being: `SELECT * FROM customers WHERE name LIKE 'A%';`. The Results tab displays the output of this query, showing a table with columns: customer_id, name, email, phone, and created_at. The results are as follows:

customer_id	name	email	phone	created_at
15	Austin Flores	user15@example.com	328.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29

The Output tab shows the execution log for the query, indicating that 4 rows were returned.

8. Electronics priced under ₹3,000:

SQL:

SELECT * FROM products WHERE category = 'Electronics' AND price < 3000;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a list of queries, with the last one being: `SELECT * FROM products WHERE category = 'Electronics' AND price < 3000;`. The Results tab displays the output of this query, showing a table with columns: product_id, name, category, price, stock_quantity, and added_on. The results are as follows:

product_id	name	category	price	stock_quantity	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 09:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38

The Output tab shows the execution log for the query, indicating that 7 rows were returned.

SQL: Mini Project

9. Product names and prices (Price Descending):

SQL:

SELECT name, price FROM products ORDER BY price DESC;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
2199 SELECT * FROM products WHERE price > 1000;
2200 #5 Products within €2,000 to €5,000:
2201 SELECT * FROM products WHERE price BETWEEN 2000 AND 5000;
2202 #6 Fetch data for specific customer IDs (Example 1, 5, 10):
2203 SELECT * FROM customers WHERE customer_id IN (1, 5, 10);
2204 #7 Identify customers whose names start with 'A':
2205 SELECT * FROM customers WHERE name LIKE 'A%';
2206 #8 Electronics priced under €3,000:
2207 SELECT * FROM products WHERE category = 'Electronics' AND price < 3000;
2208 #9 Product names and prices (Price Descending):
2209 SELECT name, price FROM products ORDER BY price DESC;
```

The Result Grid shows the output of the last query (Query 2209):

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68

The Output pane shows the execution log for the last query (Query 2209):

#	Time	Action	Message	Duration / Fetch
2149	19:44:05	SELECT * FROM customers WHERE customer_id IN (1, 5, 10) LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
2150	19:44:57	SELECT * FROM customers WHERE name LIKE 'A%'; LIMIT 0, 1000	4 row(s) returned	0.015 sec / 0.000 sec
2151	19:45:48	SELECT * FROM products WHERE category = 'Electronics' AND price < 3000 LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec
2152	19:54:52	SELECT name, price FROM products ORDER BY price DESC LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec

10. Sort by price (DESC) then by name (ASC):

SQL:

SELECT name, price FROM products ORDER BY price DESC, name ASC;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
2201 SELECT * FROM products WHERE price BETWEEN 2000 AND 5000;
2202 #6 Fetch data for specific customer IDs (Example 1, 5, 10):
2203 SELECT * FROM customers WHERE customer_id IN (1, 5, 10);
2204 #7 Identify customers whose names start with 'A':
2205 SELECT * FROM customers WHERE name LIKE 'A%';
2206 #8 Electronics priced under €3,000:
2207 SELECT * FROM products WHERE category = 'Electronics' AND price < 3000;
2208 #9 Product names and prices (Price Descending):
2209 SELECT name, price FROM products ORDER BY price DESC;
2210 #10 Sort by price (DESC) then by name (ASC):
2211 SELECT name, price FROM products ORDER BY price DESC, name ASC;
```

The Result Grid shows the output of the last query (Query 2211):

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68

The Output pane shows the execution log for the last query (Query 2211):

#	Time	Action	Message	Duration / Fetch
2150	19:44:57	SELECT * FROM customers WHERE name LIKE 'A%'; LIMIT 0, 1000	4 row(s) returned	0.015 sec / 0.000 sec
2151	19:45:48	SELECT * FROM products WHERE category = 'Electronics' AND price < 3000 LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec
2152	19:54:52	SELECT name, price FROM products ORDER BY price DESC LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
2153	19:56:30	SELECT name, price FROM products ORDER BY price DESC, name ASC LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec

Level 2: Filtering and Formatting

1. Orders with missing customer info:

SQL: SELECT * FROM orders WHERE customer_id IS NULL;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
2206 #8 Electronics priced under ₹3,000;
2207 * SELECT * FROM products WHERE category = 'Electronics' AND price < 3000;
2208 #9 Product names and prices (Price Descending):
2209 * SELECT name, price FROM products ORDER BY price DESC;
2210 #10 Sort by price (DESC) then by name (ASC):
2211 * SELECT name, price FROM products ORDER BY price DESC, name ASC;
2212
2213 # Level 2: Filtering and Formatting
2214 #1 Orders with missing customer info:
2215 * SELECT * FROM orders WHERE customer_id IS NULL;
2216
```

The Result Grid shows the results of the query in table format:

order_id	customer_id	order_date	status	total_amount
1000	NULL	2015-09-01	Completed	1000.00

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2151	19:45:48	SELECT * FROM products WHERE category = 'Electronics' AND price < 3000 LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec
2152	19:54:52	SELECT name, price FROM products ORDER BY price DESC LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
2153	19:56:30	SELECT name, price FROM products ORDER BY price DESC, name ASC LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
2154	19:58:06	SELECT * FROM orders WHERE customer_id IS NULL LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec

2. Column aliases for readability:

SQL:

SELECT name AS "Customer Name", email AS "Contact Email" FROM customers;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
2208 #9 Product names and prices (Price Descending):
2209 * SELECT name, price FROM products ORDER BY price DESC;
2210 #10 Sort by price (DESC) then by name (ASC):
2211 * SELECT name, price FROM products ORDER BY price DESC, name ASC;
2212
2213 # Level 2: Filtering and Formatting
2214 #1 Orders with missing customer info:
2215 * SELECT * FROM orders WHERE customer_id IS NULL;
2216
2217 #2 Column aliases for readability:
2218 * SELECT name AS "Customer Name", email AS "Contact Email" FROM customers;
```

The Result Grid shows the results of the query in table format:

Customer Name	Contact Email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Katlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2152	19:54:52	SELECT name, price FROM products ORDER BY price DESC LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
2153	19:56:30	SELECT name, price FROM products ORDER BY price DESC, name ASC LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
2154	19:58:06	SELECT * FROM orders WHERE customer_id IS NULL LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec
2155	19:59:02	SELECT name AS "Customer Name", email AS "Contact Email" FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

3. Calculate total value per item ordered:

SQL: SELECT order_id, product_id, (quantity * item_price) AS line_item_total
FROM order_items;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2211 SELECT name, price FROM products ORDER BY price DESC, name ASC;
2212
2213 # Level 2: Filtering and Formatting
2214 #1 Orders with missing customer info:
2215 SELECT * FROM orders WHERE customer_id IS NULL;
2216 #2 Column aliases for readability:
2217 SELECT name AS "Customer Name", email AS "Contact Email" FROM customers;
2218 #3 Calculate total value per item ordered:
2219 SELECT order_id, product_id, (quantity * item_price) AS line_item_total
2220 FROM order_items;
2221
```

The Results window shows the output of the query:

order_id	product_id	line_item_total
1	19	9414.28
2	6	532.20
3	9	2955.57
3	23	2208.99
4	35	9468.78
5	19	4707.14
5	7	9794.72
6	7	14692.08
6	37	1429.45

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
2153	19:56:30	SELECT name, price FROM products ORDER BY price DESC, name ASC LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
2154	19:58:06	SELECT * FROM orders WHERE customer_id IS NULL LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec
2155	19:59:02	SELECT name AS "Customer Name", email AS "Contact Email" FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2156	19:59:55	SELECT order_id, product_id, (quantity * item_price) AS line_item_total FROM order_items LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec

4. Combine name and phone number:

SQL: SELECT CONCAT(name, ' - ', phone) AS customer_contact FROM customers;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2213 # Level 2: Filtering and Formatting
2214 #1 Orders with missing customer info:
2215 SELECT * FROM orders WHERE customer_id IS NULL;
2216 #2 Column aliases for readability:
2217 SELECT name AS "Customer Name", email AS "Contact Email" FROM customers;
2218 #3 Calculate total value per item ordered:
2219 SELECT order_id, product_id, (quantity * item_price) AS line_item_total
2220 FROM order_items;
2221 #4 Combine name and phone number:
2222 SELECT CONCAT(name, ' - ', phone) AS customer_contact FROM customers;
2223
```

The Results window shows the output of the query:

customer_contact
Thomas Owens - 142-479-1945
Charles Grant - 9153947511
Kathin Richards - 2073473421
Christna Williams - 586-605-5061x06
David Allen - (751)456-8289x1
Mark Duke - (144)957-2811
Briana Wright - 223-833-9635
John Bryen - 045.568.0788x27
Jason Thompson - 1862659420

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
2154	19:58:06	SELECT * FROM orders WHERE customer_id IS NULL LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec
2155	19:59:02	SELECT name AS "Customer Name", email AS "Contact Email" FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2156	19:59:55	SELECT order_id, product_id, (quantity * item_price) AS line_item_total FROM order_items LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec
2157	20:00:48	SELECT CONCAT(name, ' - ', phone) AS customer_contact FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

5. Extract date part from order timestamps:

SQL:

```
SELECT order_id, DATE(order_date) AS order_day FROM orders;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a query that extracts the date part from order timestamps. The query is as follows:

```
2215 SELECT * FROM orders WHERE customer_id IS NULL;
2216 #2 Column aliases for readability:
2217 SELECT name AS "Customer Name", email AS "Contact Email" FROM customers;
2218 #3 Calculate total value per item ordered:
2219 SELECT order_id, product_id, (quantity * item_price) AS line_item_total
2220 FROM order_items;
2221 #4 Combine name and phone number:
2222 SELECT CONCAT(name, ' - ', phone) AS customer_contact FROM customers;
2223 #5 Extract date part from order timestamps:
2224 SELECT order_id, DATE(order_date) AS order_day FROM orders;
2225
```

The Result Grid shows the output of the query:

order_id	order_day
1	2025-03-02
2	2024-10-09
3	2025-05-08
4	2024-09-19
5	2025-04-08
6	2024-10-25
7	2024-07-29
8	2024-07-30
9	2025-06-10

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2155	19:59:02	SELECT name AS "Customer Name", email AS "Contact Email" FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2156	19:59:55	SELECT order_id, product_id, (quantity * item_price) AS line_item_total FROM order_items LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec
2157	20:00:48	SELECT CONCAT(name, ' - ', phone) AS customer_contact FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2158	20:01:37	SELECT order_id, DATE(order_date) AS order_day FROM orders LIMIT 0, 1000	400 row(s) returned	0.016 sec / 0.000 sec

6. Out-of-stock products:

SQL:

```
SELECT * FROM products WHERE stock_quantity = 0;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a query to find out-of-stock products. The query is as follows:

```
2217 SELECT name AS "Customer Name", email AS "Contact Email" FROM customers;
2218 #3 Calculate total value per item ordered:
2219 SELECT order_id, product_id, (quantity * item_price) AS line_item_total
2220 FROM order_items;
2221 #4 Combine name and phone number:
2222 SELECT CONCAT(name, ' - ', phone) AS customer_contact FROM customers;
2223 #5 Extract date part from order timestamps:
2224 SELECT order_id, DATE(order_date) AS order_day FROM orders;
2225 #6 Out-of-stock products:
2226 SELECT * FROM products WHERE stock_quantity = 0;
2227
```

The Result Grid shows the output of the query:

product_id	name	category	price	stock_quantity	added_on
------------	------	----------	-------	----------------	----------

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2156	19:59:55	SELECT order_id, product_id, (quantity * item_price) AS line_item_total FROM order_items LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec
2157	20:00:48	SELECT CONCAT(name, ' - ', phone) AS customer_contact FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2158	20:01:37	SELECT order_id, DATE(order_date) AS order_day FROM orders LIMIT 0, 1000	400 row(s) returned	0.016 sec / 0.000 sec
2159	20:02:26	SELECT * FROM products WHERE stock_quantity = 0 LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec

Level 3: Aggregations

1. Total number of orders:

SQL:

```
SELECT COUNT(order_id) AS total_orders FROM orders;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
2220 FROM order_items;
2221 #4 Combine name and phone number:
2222 SELECT CONCAT(name, ' ', phone) AS customer_contact FROM customers;
2223 #5 Extract date part from order timestamps:
2224 SELECT order_id, DATE(order_date) AS order_day FROM orders;
2225 #6 Out-of-stock products:
2226 SELECT * FROM products WHERE stock_quantity = 0;
2227
2228 # Level 3: Aggregations
2229 #1 Total number of orders:
2230 SELECT COUNT(order_id) AS total_orders FROM orders;
```

The Result Grid shows the output of the last query:

total_orders
400

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
2157	20:00:48	SELECT CONCAT(name, ' ', phone) AS customer_contact FROM customers LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2158	20:01:37	SELECT order_id, DATE(order_date) AS order_day FROM orders LIMIT 0, 1000	400 row(s) returned	0.016 sec / 0.000 sec
2159	20:02:26	SELECT * FROM products WHERE stock_quantity = 0 LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
2160	20:04:02	SELECT COUNT(order_id) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

2. Total revenue from all orders:

SQL:

```
SELECT SUM(total_amount) AS total_revenue FROM orders;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
2223 #5 Extract date part from order timestamps:
2224 SELECT order_id, DATE(order_date) AS order_day FROM orders;
2225 #6 Out-of-stock products:
2226 SELECT * FROM products WHERE stock_quantity = 0;
2227
2228 # Level 3: Aggregations
2229 #1 Total number of orders:
2230 SELECT COUNT(order_id) AS total_orders FROM orders;
2231 #2 Total revenue from all orders:
2232 SELECT SUM(total_amount) AS total_revenue FROM orders;
```

The Result Grid shows the output of the last query:

total_revenue
6960973.66

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
2158	20:01:37	SELECT order_id, DATE(order_date) AS order_day FROM orders LIMIT 0, 1000	400 row(s) returned	0.016 sec / 0.000 sec
2159	20:02:26	SELECT * FROM products WHERE stock_quantity = 0 LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
2160	20:04:02	SELECT COUNT(order_id) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2161	20:04:50	SELECT SUM(total_amount) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

3. Average order value:

SQL:

```
SELECT AVG(total_amount) AS avg_order_value FROM orders;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to calculate the average order value. The result grid shows the output of the query.

```
2225 #6 Out-of-stock products:
2226 SELECT * FROM products WHERE stock_quantity = 0;
2227
2228 # Level 3: Aggregations
2229 #1 Total number of orders:
2230 SELECT COUNT(order_id) AS total_orders FROM orders;
2231 #2 Total revenue from all orders:
2232 SELECT SUM(total_amount) AS total_revenue FROM orders;
2233 #3 Average order value:
2234 SELECT AVG(total_amount) AS avg_order_value FROM orders;
2235
```

avg_order_value
17402.434150

The output pane shows the execution of the query, with the message "1 row(s) returned" and a duration of 0.000 sec / 0.000 sec.

4. Number of active customers (who placed an order):

SQL:

```
SELECT COUNT(DISTINCT customer_id) AS active_customers FROM orders;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to calculate the number of active customers. The result grid shows the output of the query.

```
2227
2228 # Level 3: Aggregations
2229 #1 Total number of orders:
2230 SELECT COUNT(order_id) AS total_orders FROM orders;
2231 #2 Total revenue from all orders:
2232 SELECT SUM(total_amount) AS total_revenue FROM orders;
2233 #3 Average order value:
2234 SELECT AVG(total_amount) AS avg_order_value FROM orders;
2235 #4 Number of active customers (who placed an order):
2236 SELECT COUNT(DISTINCT customer_id) AS active_customers FROM orders;
2237
```

active_customers
30

The output pane shows the execution of the query, with the message "1 row(s) returned" and a duration of 0.000 sec / 0.000 sec.

SQL: Mini Project

5. Number of orders per customer:

SQL:

```
SELECT customer_id, COUNT(order_id) AS order_count
```

```
FROM orders
```

```
GROUP BY customer_id;
```

The screenshot displays the MySQL Workbench interface. The SQL Editor contains a query to count orders per customer. The Results window shows the output of the query, and the Output window shows the execution log.

SQL Query:

```
2230 SELECT COUNT(order_id) AS total_orders FROM orders;
2231 #2 Total revenue from all orders:
2232 SELECT SUM(total_amount) AS total_revenue FROM orders;
2233 #3 Average order value:
2234 SELECT AVG(total_amount) AS avg_order_value FROM orders;
2235 #4 Number of active customers (who placed an order):
2236 SELECT COUNT(DISTINCT customer_id) AS active_customers FROM orders;
2237 #5 Number of orders per customer:
2238 SELECT customer_id, COUNT(order_id) AS order_count
2239 FROM orders
2240 GROUP BY customer_id;
```

Result Grid:

customer_id	order_count
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10

Output:

#	Time	Action	Message	Duration / Fetch
2161	20:04:50	SELECT SUM(total_amount) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2162	20:06:58	SELECT AVG(total_amount) AS avg_order_value FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2163	20:07:39	SELECT COUNT(DISTINCT customer_id) AS active_customers FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2164	20:08:33	SELECT customer_id, COUNT(order_id) AS order_count FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

6. Total sales amount per customer:

SQL:

```
SELECT customer_id, SUM(total_amount) AS total_spent
```

```
FROM orders
```

```
GROUP BY customer_id;
```

The screenshot displays the MySQL Workbench interface. The main editor window contains a series of SQL queries, with the final query being the one specified in the project: `SELECT customer_id, SUM(total_amount) AS total_spent FROM orders GROUP BY customer_id;`. The 'Result Grid' pane below the editor shows the output of this query, displaying a table with two columns: 'customer_id' and 'total_spent'. The results are as follows:

customer_id	total_spent
1	183747.44
2	284480.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	196226.03

The 'Output' pane at the bottom shows the execution log, indicating that the query was completed successfully at 20:09:29, returning 30 rows.

SQL: Mini Project

7. Products sold per category:

SQL:

```
SELECT category, COUNT(product_id) AS total_products
```

```
FROM products
```

```
GROUP BY category;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2238 SELECT customer_id, COUNT(order_id) AS order_count
2239 FROM orders
2240 GROUP BY customer_id;
2241 #6 Total sales amount per customer:
2242 SELECT customer_id, SUM(total_amount) AS total_spent
2243 FROM orders
2244 GROUP BY customer_id;
2245 #7 Products sold per category:
2246 SELECT category, COUNT(product_id) AS total_products
2247 FROM products
2248 GROUP BY category;
```

The Results window displays the output of the last query:

category	total_products
Home	8
Clothing	13
Electronics	14
Toys	8
Books	7

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
2163	20:07:39	SELECT COUNT(DISTINCT customer_id) AS active_customers FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2164	20:08:33	SELECT customer_id, COUNT(order_id) AS order_count FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2165	20:09:29	SELECT customer_id, SUM(total_amount) AS total_spent FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2166	20:10:10	SELECT category, COUNT(product_id) AS total_products FROM products GROUP BY category LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec

SQL: Mini Project

8. Average item price per category:

SQL:

SELECT category, AVG(price) AS avg_price

FROM products

GROUP BY category;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2242 SELECT customer_id, SUM(total_amount) AS total_spent
2243 FROM orders
2244 GROUP BY customer_id;
2245 #7 Products sold per category:
2246 SELECT category, COUNT(product_id) AS total_products
2247 FROM products
2248 GROUP BY category;
2249 #8 Average item price per category:
2250 SELECT category, AVG(price) AS avg_price
2251 FROM products
2252 GROUP BY category;
```

The Results window displays the output of the third query (line 2250-2252):

category	avg_price
Home	2146.367500
Clothing	3434.581538
Electronics	2653.388571
Toys	2516.526250
Books	3167.084286

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
2164	20:08:33	SELECT customer_id, COUNT(order_id) AS order_count FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2165	20:09:29	SELECT customer_id, SUM(total_amount) AS total_spent FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2166	20:10:10	SELECT category, COUNT(product_id) AS total_products FROM products GROUP BY category LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec
2167	20:11:00	SELECT category, AVG(price) AS avg_price FROM products GROUP BY category LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec

SQL: Mini Project

9. Orders placed per day:

SQL:

```
SELECT DATE(order_date) AS order_day, COUNT(order_id) AS daily_orders
```

```
FROM orders
```

```
GROUP BY DATE(order_date);
```

The screenshot displays the MySQL Workbench interface. The SQL Editor contains the following query:

```
2246 SELECT category, COUNT(product_id) AS total_products
2247 FROM products
2248 GROUP BY category;
2249 #8 Average item price per category:
2250 SELECT category, AVG(price) AS avg_price
2251 FROM products
2252 GROUP BY category;
2253 #9 Orders placed per day:
2254 SELECT DATE(order_date) AS order_day, COUNT(order_id) AS daily_orders
2255 FROM orders
2256 GROUP BY DATE(order_date);
```

The Results window shows the output of the last query, displaying two columns: `order_day` and `daily_orders`. The data is as follows:

order_day	daily_orders
2025-03-02	3
2024-10-09	2
2025-05-08	2
2024-09-19	3
2025-04-08	3
2024-10-25	3
2024-07-29	2
2024-07-30	2
2025-06-10	2

The Output window at the bottom shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
2165	20:09:29	SELECT customer_id, SUM(total_amount) AS total_spent FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2166	20:10:10	SELECT category, COUNT(product_id) AS total_products FROM products GROUP BY category LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec
2167	20:11:00	SELECT category, AVG(price) AS avg_price FROM products GROUP BY category LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec
2168	20:12:54	SELECT DATE(order_date) AS order_day, COUNT(order_id) AS daily_orders FROM orders GROUP BY DATE...	232 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

10. Total payments per method:

SQL:

SELECT method, SUM(amount_paid) AS total_received

FROM payments

GROUP BY method;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
2250 SELECT category, AVG(price) AS avg_price
2251 FROM products
2252 GROUP BY category;
2253 #9 Orders placed per day:
2254 SELECT DATE(order_date) AS order_day, COUNT(order_id) AS daily_orders
2255 FROM orders
2256 GROUP BY DATE(order_date);
2257 #10 Total payments per method:
2258 SELECT method, SUM(amount_paid) AS total_received
2259 FROM payments
2260 GROUP BY method;
```

The Result Grid shows the output of the last query:

method	total_received
Credit Card	1754603.10
Net Banking	1638383.90
UPI	1617408.78
Debit Card	1930577.88

The Output panel shows the execution log:

#	Time	Action	Message	Duration / Fetch
2166	20:10:10	SELECT category, COUNT(product_id) AS total_products FROM products GROUP BY category LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec
2167	20:11:00	SELECT category, AVG(price) AS avg_price FROM products GROUP BY category LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec
2168	20:12:54	SELECT DATE(order_date) AS order_day, COUNT(order_id) AS daily_orders FROM orders GROUP BY DATE...	232 row(s) returned	0.000 sec / 0.000 sec
2169	20:13:38	SELECT method, SUM(amount_paid) AS total_received FROM payments GROUP BY method LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

Level 4: Multi-Table Queries (JOINS)

1. Order details with customer name:

SQL:

```
SELECT o.order_id, c.name, o.order_date, o.total_amount
```

```
FROM orders o
```

```
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2255 FROM orders
2256 GROUP BY DATE(order_date);
2257 #18 Total payments per method:
2258 SELECT method, SUM(amount_paid) AS total_received
2259 FROM payments
2260 GROUP BY method;
2261 # Level 4: Multi-Table Queries (JOINS)
2262 #1 Order details with customer name:
2263 SELECT o.order_id, c.name, o.order_date, o.total_amount
2264 FROM orders o
2265 INNER JOIN customers c ON o.customer_id = c.customer_id;
```

The Results Grid shows the output of the query:

order_id	name	order_date	total_amount
14	Thomas Owens	2024-09-24 21:21:38	15803.34
17	Thomas Owens	2024-08-19 21:17:57	11173.77
61	Thomas Owens	2024-12-26 23:21:58	13053.00
76	Thomas Owens	2025-01-14 22:59:54	23506.81
92	Thomas Owens	2024-09-26 11:33:58	834.75
109	Thomas Owens	2025-03-17 04:56:18	12190.14
127	Thomas Owens	2025-06-10 18:01:32	20160.64
135	Thomas Owens	2025-03-11 22:58:23	8891.79
144	Thomas Owens	2024-09-19 09:18:22	12093.54

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
2167	20:11:00	SELECT category, AVG(price) AS avg_price FROM products GROUP BY category LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec
2168	20:12:54	SELECT DATE(order_date) AS order_day, COUNT(order_id) AS daily_orders FROM orders GROUP BY DATE(order_date) LIMIT 0, 1000	232 row(s) returned	0.000 sec / 0.000 sec
2169	20:13:38	SELECT method, SUM(amount_paid) AS total_received FROM payments GROUP BY method LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
2170	20:18:16	SELECT o.order_id, c.name, o.order_date, o.total_amount FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id	400 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

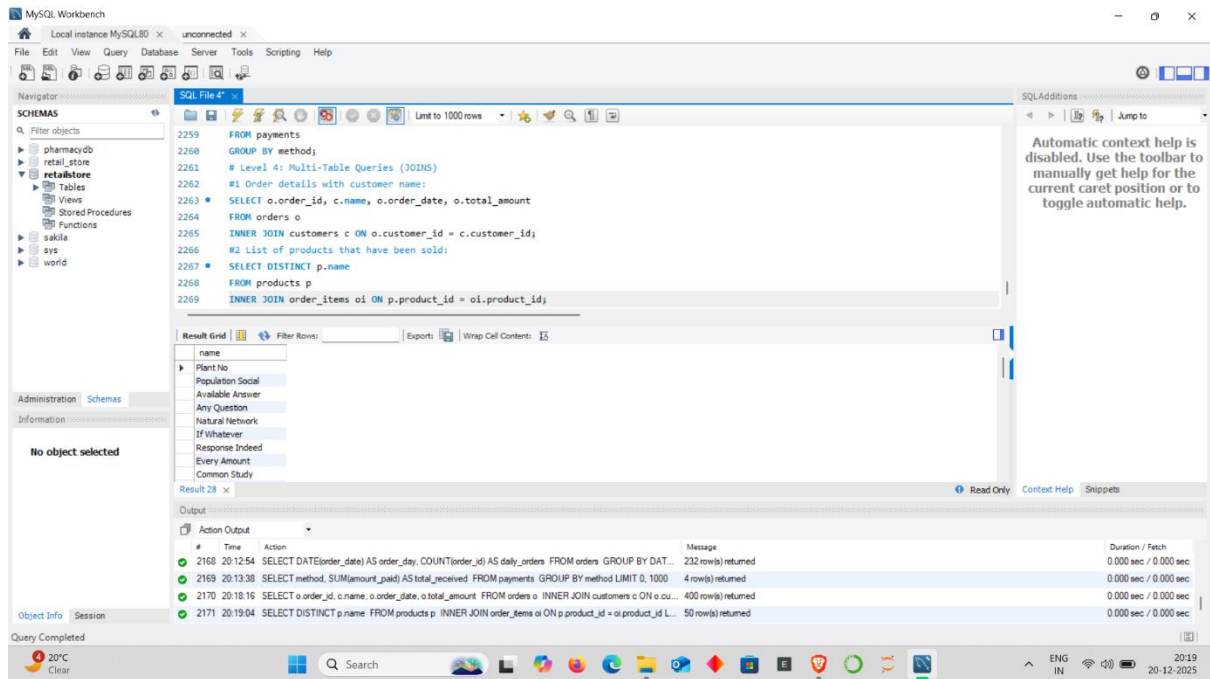
2. List of products that have been sold:

SQL:

SELECT DISTINCT p.name

FROM products p

INNER JOIN order_items oi ON p.product_id = oi.product_id;



SQL: Mini Project

3. Orders with payment method:

SQL:

```
SELECT o.order_id, o.total_amount, p.method
```

```
FROM orders o
```

```
INNER JOIN payments p ON o.order_id = p.order_id;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
2263 SELECT o.order_id, c.name, o.order_date, o.total_amount
2264 FROM orders o
2265 INNER JOIN customers c ON o.customer_id = c.customer_id;
2266 #2 List of products that have been sold:
2267 SELECT DISTINCT p.name
2268 FROM products p
2269 INNER JOIN order_items oi ON p.product_id = oi.product_id;
2270 #3 Orders with payment method:
2271 SELECT o.order_id, o.total_amount, p.method
2272 FROM orders o
2273 INNER JOIN payments p ON o.order_id = p.order_id;
```

The Results tab shows the output of the third query, displaying a table with columns: order_id, total_amount, and method.

order_id	total_amount	method
1	9414.28	Credit Card
2	532.20	Net Banking
3	5164.56	Credit Card
4	9469.78	UPI
5	1450.186	UPI
6	31050.17	UPI
7	3043.67	UPI
8	32714.06	Net Banking
9	24215.20	Net Banking

The Output tab shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
2169	20:13:38	SELECT method, SUM(amount_paid) AS total_received FROM payments GROUP BY method LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
2170	20:18:16	SELECT o.order_id, c.name, o.order_date, o.total_amount FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id	400 row(s) returned	0.000 sec / 0.000 sec
2171	20:19:04	SELECT DISTINCT p.name FROM products p INNER JOIN order_items oi ON p.product_id = oi.product_id	50 row(s) returned	0.000 sec / 0.000 sec
2172	20:20:07	SELECT o.order_id, o.total_amount, p.method FROM orders o INNER JOIN payments p ON o.order_id = p.order_id	400 row(s) returned	0.015 sec / 0.000 sec

SQL: Mini Project

4. All customers and their orders:

SQL:

```
SELECT c.name, o.order_id
```

```
FROM customers c
```

```
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2267 SELECT DISTINCT p.name
2268 FROM products p
2269 INNER JOIN order_items oi ON p.product_id = oi.product_id;
2270 #3 Orders with payment method:
2271 SELECT o.order_id, o.total_amount, p.method
2272 FROM orders o
2273 INNER JOIN payments p ON o.order_id = p.order_id;
2274 #4 All customers and their orders:
2275 SELECT c.name, o.order_id
2276 FROM customers c
2277 LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

The Results window shows the output of the last query (Query 2277):

name	order_id
Thomas Owens	14
Thomas Owens	17
Thomas Owens	61
Thomas Owens	76
Thomas Owens	92
Thomas Owens	109
Thomas Owens	127
Thomas Owens	135
Thomas Owens	144

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
2170	20:18:16	SELECT o.order_id, c.name, o.order_date, o.total_amount FROM orders o INNER JOIN customers c ON o.cu...	400 row(s) returned	0.000 sec / 0.000 sec
2171	20:19:04	SELECT DISTINCT p.name FROM products p INNER JOIN order_items oi ON p.product_id = oi.product_id L...	50 row(s) returned	0.000 sec / 0.000 sec
2172	20:20:07	SELECT o.order_id, o.total_amount, p.method FROM orders o INNER JOIN payments p ON o.order_id = p.or...	400 row(s) returned	0.015 sec / 0.000 sec
2173	20:20:57	SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id LIM...	400 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

5. All products and quantity sold:

SQL:

```
SELECT p.name, SUM(oi.quantity) AS total_quantity_sold
```

```
FROM products p
```

```
LEFT JOIN order_items oi ON p.product_id = oi.product_id
```

```
GROUP BY p.name;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2272 FROM orders o
2273 INNER JOIN payments p ON o.order_id = p.order_id;
2274 #4 All customers and their orders:
2275 SELECT c.name, o.order_id
2276 FROM customers c
2277 LEFT JOIN orders o ON c.customer_id = o.customer_id;
2278 #5 All products and quantity sold:
2279 SELECT p.name, SUM(oi.quantity) AS total_quantity_sold
2280 FROM products p
2281 LEFT JOIN order_items oi ON p.product_id = oi.product_id
2282 GROUP BY p.name;
```

The Results Grid shows the output of the query:

name	total_quantity_sold
Plant No	43
Population Social	33
Available Answer	47
Any Question	30
Natural Network	44
If Whatever	54
Response Indexed	39
Every Amount	61
Common Study	57

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2171	20:19:04	SELECT DISTINCT p.name FROM products p INNER JOIN order_items oi ON p.product_id = oi.product_id L...	50 row(s) returned	0.000 sec / 0.000 sec
2172	20:20:07	SELECT o.order_id, o.total_amount, p.method FROM orders o INNER JOIN payments p ON o.order_id = p.or...	400 row(s) returned	0.015 sec / 0.000 sec
2173	20:20:57	SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id LIM...	400 row(s) returned	0.000 sec / 0.000 sec
2174	20:21:40	SELECT p.name, SUM(oi.quantity) AS total_quantity_sold FROM products p LEFT JOIN order_items oi ON p...	50 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

6. Payments with no matching orders (Right Join):

SQL:

```
SELECT p.*, o.order_id
```

```
FROM orders o
```

```
RIGHT JOIN payments p ON o.order_id = p.order_id;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
2276 FROM customers c
2277 LEFT JOIN orders o ON c.customer_id = o.customer_id;
2278 #5 All products and quantity sold:
2279 SELECT p.name, SUM(oi.quantity) AS total_quantity_sold
2280 FROM products p
2281 LEFT JOIN order_items oi ON p.product_id = oi.product_id
2282 GROUP BY p.name;
2283 #6 Payments with no matching orders (Right Join):
2284 SELECT p.*, o.order_id
2285 FROM orders o
2286 RIGHT JOIN payments p ON o.order_id = p.order_id;
```

The Results tab shows the output of the last query, which is a right join between payments and orders. The table has 6 columns: payment_id, order_id, payment_date, amount_paid, method, and order_id. The data is as follows:

payment_id	order_id	payment_date	amount_paid	method	order_id
1	1	2025-03-02 08:16:11	9414.28	Credit Card	1
2	2	2024-10-09 19:00:21	532.20	Net Banking	2
3	3	2025-05-08 00:55:27	5164.56	Credit Card	3
4	4	2024-09-19 22:28:13	9469.78	UPI	4
5	5	2025-04-08 18:26:06	1450.186	UPI	5
6	6	2024-10-25 08:16:59	31050.17	UPI	6
7	7	2024-07-29 12:45:47	3043.67	UPI	7
8	8	2024-07-30 23:42:49	32714.06	Net Banking	8
9	9	2025-06-10 17:36:25	24219.20	Net Banking	9

The Output tab shows the execution log with the following messages:

- 2172 20:20:07 SELECT o.order_id, o.total_amount, p.method FROM orders o INNER JOIN payments p ON o.order_id = p.order_id; 400 row(s) returned 0.015 sec / 0.000 sec
- 2173 20:20:57 SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id LIMIT 10; 400 row(s) returned 0.000 sec / 0.000 sec
- 2174 20:21:40 SELECT p.name, SUM(oi.quantity) AS total_quantity_sold FROM products p LEFT JOIN order_items oi ON p.product_id = oi.product_id GROUP BY p.name; 50 row(s) returned 0.000 sec / 0.000 sec
- 2175 20:22:25 SELECT p.*, o.order_id FROM orders o RIGHT JOIN payments p ON o.order_id = p.order_id LIMIT 0, 1000; 400 row(s) returned 0.000 sec / 0.000 sec

SQL: Mini Project

7. Combine Customer, Order, and Payment:

SQL:

```
SELECT c.name, o.order_id, o.total_amount, p.amount_paid, p.method
```

```
FROM customers c
```

```
INNER JOIN orders o ON c.customer_id = o.customer_id
```

```
INNER JOIN payments p ON o.order_id = p.order_id;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2281 LEFT JOIN order_items oi ON p.product_id = oi.product_id
2282 GROUP BY p.name;
2283 #6 Payments with no matching orders (Right Join):
2284 SELECT p.*, o.order_id
2285 FROM orders o
2286 RIGHT JOIN payments p ON o.order_id = p.order_id
2287 #7 Combine Customer, Order, and Payment:
2288 SELECT c.name, o.order_id, o.total_amount, p.amount_paid, p.method
2289 FROM customers c
2290 INNER JOIN orders o ON c.customer_id = o.customer_id
2291 INNER JOIN payments p ON o.order_id = p.order_id
```

The Results Grid shows the following data:

name	order_id	total_amount	amount_paid	method
Thomas Owens	14	15803.34	15803.34	LPI
Thomas Owens	17	11173.77	11173.77	LPI
Thomas Owens	61	13053.00	13053.00	Net Banking
Thomas Owens	76	23506.81	23506.81	Credit Card
Thomas Owens	92	834.75	834.75	Net Banking
Thomas Owens	109	12190.14	12190.14	LPI
Thomas Owens	127	20160.64	20160.64	Debit Card
Thomas Owens	135	8891.79	8891.79	Debit Card
Thomas Owens	144	12093.54	12093.54	LPI

The Output pane shows the execution log with the following messages:

```
2173 20:20:57 SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id LIMIT 0, 1000 400 row(s) returned 0.000 sec / 0.000 sec
2174 20:21:40 SELECT p.name, SUM(oi.quantity) AS total_quantity_sold FROM products p LEFT JOIN order_items oi ON p.product_id = oi.product_id LIMIT 0, 1000 50 row(s) returned 0.000 sec / 0.000 sec
2175 20:22:25 SELECT p.*, o.order_id FROM orders o RIGHT JOIN payments p ON o.order_id = p.order_id LIMIT 0, 1000 400 row(s) returned 0.000 sec / 0.000 sec
2176 20:23:11 SELECT c.name, o.order_id, o.total_amount, p.amount_paid, p.method FROM customers c INNER JOIN orders o ON c.customer_id = o.customer_id INNER JOIN payments p ON o.order_id = p.order_id LIMIT 0, 1000 400 row(s) returned 0.000 sec / 0.000 sec
```

Level 5: Subqueries

1. Products priced above average:

SQL:

```
SELECT * FROM products
```

```
WHERE price > (SELECT AVG(price) FROM products);
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
#7 Combine Customer, Order, and Payment:
2288 SELECT c.name, o.order_id, o.total_amount, p.amount_paid, p.method
2289 FROM customers c
2290 INNER JOIN orders o ON c.customer_id = o.customer_id
2291 INNER JOIN payments p ON o.order_id = p.order_id;
2292
# Level 5: Subqueries
2293
2294
2295 #1 Products priced above average:
2296 SELECT * FROM products
2297 WHERE price > (SELECT AVG(price) FROM products);
```

The Results Grid shows the following data:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
4	Any Question	Clothing	4759.38	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Aids	Electronics	4017.01	19	2025-02-14 03:38:06
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01

The Output pane shows the execution of the query, with the following messages:

```
2174 20:21:40 SELECT p.name, SUM(p.quantity) AS total_quantity_sold FROM products p LEFT JOIN order_items o ON p... 50 row(s) returned 0.000 sec / 0.000 sec
2175 20:22:25 SELECT p.*, o.order_id FROM orders o RIGHT JOIN payments p ON o.order_id = p.order_id LIMIT 0, 1000 400 row(s) returned 0.000 sec / 0.000 sec
2176 20:23:11 SELECT c.name, o.order_id, o.total_amount, p.amount_paid, p.method FROM customers c INNER JOIN orde... 400 row(s) returned 0.000 sec / 0.000 sec
2177 20:24:21 SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products) LIMIT 0, 1000 27 row(s) returned 0.000 sec / 0.000 sec
```

SQL: Mini Project

2. Customers who placed at least one order:

SQL:

SELECT * FROM customers

WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2290 INNER JOIN orders o ON c.customer_id = o.customer_id
2291 INNER JOIN payments p ON o.order_id = p.order_id;
2292
2293 # Level 5: Subqueries
2294
2295 #1 Products priced above average:
2296 SELECT * FROM products
2297 WHERE price > (SELECT AVG(price) FROM products);
2298 #2 Customers who placed at least one order:
2299 SELECT * FROM customers
2300 WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```

The Results Grid shows the output of the query, displaying a table with columns: customer_id, name, email, phone, and created_at. The table contains 9 rows of data.

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Karin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-9061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-6289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51

The Output tab shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
2175	20:22:25	SELECT p.*, o.order_id FROM orders o RIGHT JOIN payments p ON o.order_id = p.order_id LIMIT 0, 1000	400 row(s) returned	0.000 sec / 0.000 sec
2176	20:23:11	SELECT c.name, o.order_id, o.total_amount, p.amount_paid, p.method FROM customers c INNER JOIN orde...	400 row(s) returned	0.000 sec / 0.000 sec
2177	20:24:21	SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products) LIMIT 0, 1000	27 row(s) returned	0.000 sec / 0.000 sec
2178	20:26:32	SELECT * FROM customers WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders) LIMIT ...	30 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

3. Orders above the customer's own average:

SQL:

```
SELECT * FROM orders o1
```

```
WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o2.customer_id =  
o1.customer_id);
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2293 * Level 5: Subqueries  
2294  
2295 #1 Products priced above average:  
2296 * SELECT * FROM products  
2297 WHERE price > (SELECT AVG(price) FROM products);  
2298 #2 Customers who placed at least one order:  
2299 * SELECT * FROM customers  
2300 WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);  
2301 #3 Orders above the customer's own average:  
2302 * SELECT * FROM orders o1  
2303 WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o2.customer_id = o1.customer_id);
```

The Results Grid shows the following data:

order_id	customer_id	order_date	status	total_amount
6	29	2024-10-25 07:33:59	Cancelled	31050.17
8	19	2024-07-30 22:49:49	Cancelled	32714.06
9	6	2025-06-10 17:00:25	Pending	24019.20
10	28	2025-02-16 12:45:59	Delivered	24342.52
14	1	2024-09-24 21:21:38	Shipped	15803.34
16	5	2024-09-09 03:00:40	Shipped	22856.73
18	13	2024-06-15 12:57:04	Cancelled	32001.24
21	22	2025-03-12 18:53:46	Pending	25364.11
22	20	2025-02-21 11:21:06	Shipped	21281.44

The Output tab shows the execution of the query, with the following results:

#	Time	Action	Message	Duration / Fetch
2176	20:23:11	SELECT o.name, o.order_id, o.total_amount, p.amount_paid, p.method FROM customers c INNER JOIN orde...	400 row(s) returned	0.000 sec / 0.000 sec
2177	20:24:21	SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products) LIMIT 0, 1000	27 row(s) returned	0.000 sec / 0.000 sec
2178	20:26:32	SELECT * FROM customers WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders) LIMIT ...	30 row(s) returned	0.000 sec / 0.000 sec
2179	20:28:29	SELECT * FROM orders o1 WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o...	194 row(s) returned	0.016 sec / 0.000 sec

SQL: Mini Project

4. Inactive customers (No orders):

SQL:

```
SELECT * FROM customers
```

```
WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2296 SELECT * FROM products
2297 WHERE price > (SELECT AVG(price) FROM products);
2298 #2 Customers who placed at least one order:
2299 SELECT * FROM customers
2300 WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
2301 #3 Orders above the customer's own average:
2302 SELECT * FROM orders o1
2303 WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o2.customer_id = o1.customer_id);
2304 #4 Inactive customers (No orders):
2305 SELECT * FROM customers
2306 WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
```

The Results window shows the execution of the queries. The first query returns 27 rows. The second query returns 30 rows. The third query returns 194 rows. The fourth query returns 0 rows.

#	Time	Action	Message	Duration / Fetch
2177	20:24:21	SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products) LIMIT 0, 1000	27 row(s) returned	0.000 sec / 0.000 sec
2178	20:26:32	SELECT * FROM customers WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders) LIMIT ...	30 row(s) returned	0.000 sec / 0.000 sec
2179	20:28:29	SELECT * FROM orders o1 WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o...	194 row(s) returned	0.016 sec / 0.000 sec
2180	20:29:13	SELECT * FROM customers WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders) LI...	0 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

5. Products never ordered:

SQL:

```
SELECT * FROM products
```

```
WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items);
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2299 SELECT * FROM customers
2300 WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
2301 #3 Orders above the customer's own average:
2302 SELECT * FROM orders o1
2303 WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o2.customer_id = o1.customer_id);
2304 #4 Inactive customers (No orders):
2305 SELECT * FROM customers
2306 WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
2307 #5 Products never ordered:
2308 SELECT * FROM products
2309 WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items);
```

The Results Grid shows the execution of the query, displaying the following columns: product_id, name, category, price, stock_quantity, and added_on. The results are as follows:

product_id	name	category	price	stock_quantity	added_on
10285	10285	10285	10285	10285	10285

The Output pane shows the execution log with the following messages:

```
2178 20:26:32 SELECT * FROM customers WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders) LIMIT ... 30 row(s) returned
2179 20:28:29 SELECT * FROM orders o1 WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o2.customer_id = o1.customer_id) ... 194 row(s) returned
2180 20:29:13 SELECT * FROM customers WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders) L... 0 row(s) returned
2181 20:30:26 SELECT * FROM products WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items) L... 0 row(s) returned
```

SQL: Mini Project

6. Highest value order per customer:

SQL:

```
SELECT customer_id, MAX(total_amount)
```

```
FROM orders
```

```
GROUP BY customer_id;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2303 WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o2.customer_id = o1.customer_id);
2304 #4 Inactive customers (No orders):
2305 SELECT * FROM customers
2306 WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
2307 #5 Products never ordered:
2308 SELECT * FROM products
2309 WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items);
2310 #6 Highest value order per customer:
2311 SELECT customer_id, MAX(total_amount)
2312 FROM orders
2313 GROUP BY customer_id;
```

The Results window shows the output of the query:

customer_id	MAX(total_amount)
1	32015.16
2	42096.04
3	41679.11
4	25747.34
5	39921.78
6	39003.19
7	28589.04
8	36147.09
9	21414.44

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
2179	20:28:29	SELECT * FROM orders o1 WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o2.customer_id = o1.customer_id);	194 row(s) returned	0.016 sec / 0.000 sec
2180	20:29:13	SELECT * FROM customers WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);	0 row(s) returned	0.000 sec / 0.000 sec
2181	20:30:26	SELECT * FROM products WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items);	0 row(s) returned	0.000 sec / 0.000 sec
2182	20:32:02	SELECT customer_id, MAX(total_amount) FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec

SQL: Mini Project

7. Highest Order Per Customer (Including Names):

SQL:

```
SELECT c.name, MAX(o.total_amount) as max_order
```

```
FROM customers c
```

```
JOIN orders o ON c.customer_id = o.customer_id
```

```
GROUP BY c.name;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2308 SELECT * FROM products
2309 WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items);
2310 #6 Highest value order per customer:
2311 SELECT customer_id, MAX(total_amount)
2312 FROM orders
2313 GROUP BY customer_id;
2314 #7 Highest Order Per Customer (Including Names):
2315 SELECT c.name, MAX(o.total_amount) as max_order
2316 FROM customers c
2317 JOIN orders o ON c.customer_id = o.customer_id
2318 GROUP BY c.name;
```

The Results Grid shows the output of the query:

name	max_order
Thomas Owens	32015.16
Charles Grant	42056.04
Katlin Richards	41679.11
Christna Williams	25747.34
David Allen	39921.79
Mark Duke	39003.19
Briana Wright	28589.04
John Bryan	36147.09
Jason Thompson	21414.44

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
2180	20:29:13	SELECT * FROM customers WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders) L...	0 row(s) returned	0.000 sec / 0.000 sec
2181	20:30:26	SELECT * FROM products WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items) L...	0 row(s) returned	0.000 sec / 0.000 sec
2182	20:32:02	SELECT customer_id, MAX(total_amount) FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2183	20:32:58	SELECT c.name, MAX(o.total_amount) as max_order FROM customers c JOIN orders o ON c.customer_id = o...	30 row(s) returned	0.016 sec / 0.000 sec

Level 6: Set Operations

1. Customers who ordered OR reviewed:

SQL:

```
SELECT customer_id FROM orders
```

UNION

```
SELECT customer_id FROM product_reviews;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
2315 SELECT c.name, MAX(o.total_amount) as max_order
2316 FROM customers c
2317 JOIN orders o ON c.customer_id = o.customer_id
2318 GROUP BY c.name;
2319
2320 # Level 6: Set Operations
2321
2322 #1 Customers who ordered OR reviewed:
2323 SELECT customer_id FROM orders
2324 UNION
2325 SELECT customer_id FROM product_reviews;
```

The Results grid shows the output of the query, displaying a list of customer IDs. The first column is labeled 'customer_id' and contains values 1 through 9.

The Output tab shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
2181	20:30:26	SELECT * FROM products WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items) L...	0 row(s) returned	0.000 sec / 0.000 sec
2182	20:32:02	SELECT customer_id, MAX(total_amount) FROM orders GROUP BY customer_id LIMIT 0,1000	30 row(s) returned	0.000 sec / 0.000 sec
2183	20:32:58	SELECT c.name, MAX(o.total_amount) as max_order FROM customers c JOIN orders o ON c.customer_id = o...	30 row(s) returned	0.016 sec / 0.000 sec
2184	20:33:59	SELECT customer_id FROM orders UNION SELECT customer_id FROM product_reviews	30 row(s) returned	0.000 sec / 0.000 sec

The bottom status bar indicates 'Query Completed' and the system clock shows 20:34 on 20-12-2025.

SQL: Mini Project

2. Customers who ordered AND reviewed (using IN because Intersect is often unsupported):

SQL:

```
SELECT DISTINCT customer_id FROM orders
```

```
WHERE customer_id IN (SELECT customer_id FROM product_reviews);
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
2319
2320 # Level 6: Set Operations
2321
2322 #1 Customers who ordered OR reviewed:
2323 SELECT customer_id FROM orders
2324 UNION
2325 SELECT customer_id FROM product_reviews;
2326
2327 #2 Customers who ordered AND reviewed (using IN because Intersect is often unsupported):
2328 SELECT DISTINCT customer_id FROM orders
2329 WHERE customer_id IN (SELECT customer_id FROM product_reviews);
```

The Result Grid shows the results of the query, displaying a list of customer IDs:

customer_id
1
2
4
6
7
9
10
11
13

The Output tab shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
2182	20:32:02	SELECT customer_id, MAX(total_amount) FROM orders GROUP BY customer_id LIMIT 0, 1000	30 row(s) returned	0.000 sec / 0.000 sec
2183	20:32:58	SELECT c.name, MAX(o.total_amount) as max_order FROM customers c JOIN orders o ON c.customer_id = o.customer_id	30 row(s) returned	0.016 sec / 0.000 sec
2184	20:33:59	SELECT customer_id FROM orders UNION SELECT customer_id FROM product_reviews	30 row(s) returned	0.000 sec / 0.000 sec
2185	20:36:01	SELECT DISTINCT customer_id FROM orders WHERE customer_id IN (SELECT customer_id FROM product_reviews)	23 row(s) returned	0.015 sec / 0.000 sec